Техническое задание

«P2P BIG Torrent»

(Безопасный P2P-распространитель файлов для организации)

---

1. Введение

1.1. Наименование проекта

P2P BIG Torrent (далее — Система).

1.2. Цель проекта

Разработать веб-приложение для внутренней сети организации, позволяющее авторизованным сотрудникам ускоренно загружать большие файлы (образы виртуальных машин, ISO-образы, дампы баз данных) за счёт одновременной загрузки с нескольких источников (P2P). После получения файла браузер пользователя автоматически становится распространителем (сидом) для других пользователей. Администратор организации управляет списком разрешённых файлов.

1.3. Задачи проекта

- Реализовать веб-интерфейс на Bootstrap.
- Обеспечить регистрацию и авторизацию пользователей с разделением ролей (администратор, пользователь).
- Реализовать загрузку файлов администратором, разбиение на куски, вычисление хэшей кусков.
- Разработать собственный WebTorrent-подобный клиент на JavaScript (без использования готовых торрент-библиотек) для P2P-обмена через WebRTC DataChannel.
- Реализовать серверный трекер на Flask для координации пиров.
- Обеспечить хранение данных в SQLite через SQLAlchemy.
- Создать REST API для взаимодействия клиента с сервером.

---

2. Технологический стек

Компонент	Технология	Примечание
Бэкенд	Python 3.12 / Flask	Веб-фреймворк
База данных	SQLite	Лёгкая, файловая
ORM	SQLAlchemy	Работа с моделями
Фронтенд	HTML5 / Bootstrap 5	Адаптивный интерфейс
P2P-клиент (браузер)	Чистый JavaScript (WebRTC, DataChannel, File API, Crypto)	**Запрещены** WebTorrent.js, webtorrent, bittorrent-tracker и аналоги
API	REST (JSON)	Собственная реализация
Хостинг	Локальный сервер организации или VPS	Требуется HTTPS для WebRTC

---

3. Требования к системе

3.1. Функциональные требования

3.1.1. Управление пользователями

- Регистрация нового пользователя (email, пароль, имя).
- Авторизация по email/паролю (сессии или JWT).
- Разделение ролей: **администратор** и **пользователь**.
- Только авторизованные пользователи имеют доступ к списку файлов и скачиванию.

3.1.2. Управление файлами (администратор)

- Загрузка файла через веб-форму (название, опционально описание).
- Автоматическое разбиение файла на куски фиксированного размера (например, 1 МБ).
- Вычисление SHA-256 хэша для каждого куска.
- Сохранение метаинформации о файле в БД.
- Физическое сохранение файла на сервере для начальной раздачи.
- Возможность удалить файл (удаляется запись из БД и физический файл, раздача прекращается).

3.1.3. P2P-скачивание (пользователь)

- Просмотр списка доступных файлов с информацией (имя, размер, количество текущих сидов).
- Инициация скачивания выбранного файла.
- Работа самописного WebTorrent-клиента в браузере:
  - Получение метаинформации о файле (размер, количество кусков, хэши кусков) через REST API.
  - Подключение к трекеру (WebSocket или HTTP polling) для координации.
  - Обнаружение других пиров, имеющих нужные куски.
  - Установка WebRTC DataChannel с пирами.
  - Запрос и получение недостающих кусков.
  - Проверка целостности каждого куска по хэшу.
  - Сборка файла в Blob и сохранение на диск пользователя.
  - После полной загрузки — уведомление трекера о том, что пользователь теперь сид (имеет все куски).
- Отображение прогресса загрузки, скорости, количества активных пиров.

3.1.4. Трекер (серверная координация)

- Приём сообщений от пиров: «у меня есть куски X, Y, Z файла F».
- Хранение в БД информации о том, какие куски каких файлов есть у каждого пира.
- При запросе от клиента возвращать список других пиров, у которых есть запрашиваемые куски.
- Очистка устаревших пиров (тайм-аут).

3.1.5. REST API (обязательные эндпоинты)

- `GET /api/files` — список файлов (только для авторизованных).
- `GET /api/files/<id>` — метаданные файла (размер, число кусков, список хэшей).
- `POST /api/upload` — загрузка нового файла (только админ).
- `DELETE /api/files/<id>` — удаление файла (админ).
- `POST /api/announce` — трекер-эндпоинт (пир сообщает свои куски, получает список других пиров).

3.1.6. Безопасность

- Все запросы к API, кроме регистрации/логина, требуют аутентификации.
- Передача файловых кусков происходит только через WebRTC (DTLS шифрование).
- Сервер не отдаёт файлы напрямую по HTTP (кроме служебной загрузки администратором).
- Рекомендуется развёртывание по HTTPS.

3.2. Нефункциональные требования

- **Производительность:** поддержка одновременной загрузки файлов до 10 ГБ (на тестовых данных).
- **Надёжность:** восстановление загрузки после обрыва соединения (повтор запросов кусков).
- **Масштабируемость:** при увеличении числа пользователей нагрузка на сервер снижается за счёт P2P.
- **Код:** соблюдение PEP 8, разбиение на модули, использование классов.

Ниже представлен доработанный и уточнённый раздел «4. Архитектура и модели данных» в формате Markdown. Внесены незначительные исправления и дополнения для большей точности и соответствия полному системному промту. Вы можете скопировать этот текст в свой документ ТЗ.

---

4. Архитектура и модели данных

4.1. Общая архитектура

- Браузер пользователя загружает веб-страницу (Flask + Bootstrap).
- Через REST API получает список файлов и метаданные.
- Самописный JavaScript-клиент (WebTorrent-подобный) использует WebRTC DataChannel для обмена кусками с другими пирами.
- Серверный трекер на Flask принимает announce-запросы и хранит состояние пиров в БД.
- Администратор через веб-интерфейс загружает новые файлы на сервер.
- **Примечание:** Сервер также выступает в роли постоянного сида для всех загруженных файлов (peer_id = "server").

4.2. Модели данных (SQLAlchemy)

```
# models.py

from datetime import datetime
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
```

```python
class User(db.Model):
    __tablename__ = 'users'
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(128), nullable=False)
    role = db.Column(db.String(20), default='user')  # 'admin' или 'user'

class File(db.Model):
    __tablename__ = 'files'
    id = db.Column(db.Integer, primary_key=True)
    filename = db.Column(db.String(255), nullable=False)
    file_size = db.Column(db.Integer, nullable=False)    # в байтах
    piece_length = db.Column(db.Integer, nullable=False) # размер куска в
байтах
    piece_hashes = db.Column(db.JSON, nullable=False)    # список SHA-256
хэшей (строки в hex)
    uploader_id = db.Column(db.Integer, db.ForeignKey('users.id'))
    uploaded_at = db.Column(db.DateTime, default=datetime.utcnow)
    uploader = db.relationship('User', backref='uploaded_files')

class Peer(db.Model):
    __tablename__ = 'peers'
    id = db.Column(db.Integer, primary_key=True)
    peer_id = db.Column(db.String(64), unique=True, nullable=False)  #
генерируется клиентом или "server"
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=True)
# для обычных пиров
    file_id = db.Column(db.Integer, db.ForeignKey('files.id'))
    has_all_pieces = db.Column(db.Boolean, default=False)
    last_seen = db.Column(db.DateTime, default=datetime.utcnow)

class Piece(db.Model):
    __tablename__ = 'pieces'
    id = db.Column(db.Integer, primary_key=True)
    file_id = db.Column(db.Integer, db.ForeignKey('files.id'))
    peer_id = db.Column(db.Integer, db.ForeignKey('peers.id'))
    piece_index = db.Column(db.Integer, nullable=False)  # индекс куска от 0
    has_piece = db.Column(db.Boolean, default=True)      # есть ли этот кусок
у пира
    __table_args__ = (db.UniqueConstraint('file_id', 'peer_id', 'piece_index',
name='unique_piece'),)
```

4.3. Схема взаимодействия компонентов

1. Загрузка файла администратором

- Администратор загружает файл через веб-форму (`/admin/upload`).
- Сервер:
  - Сохраняет исходный файл во временную директорию.
  - Разбивает его на куски фиксированного размера (например, 1 МБ).
  - Для каждого куска вычисляет SHA-256 хэш.
  - Сохраняет куски на диск в папку `uploads/pieces/<file_id>/` .
  - Создаёт запись в таблице `File` (поля: `filename` , `file_size` , `piece_length` , `piece_hashes` (JSON-массив хэшей), `uploader_id`).
  - Создаёт запись в таблице `Peer` для серверного пира: `peer_id = "server"` , `file_id` , `has_all_pieces = True` .
  - Для каждого куска создаёт запись в таблице `Piece` : `file_id` , `peer_id` (сервера), `piece_index` , `has_piece = True` .

2. Скачивание первым пользователем (Пользователь А)

- Пользователь А видит список файлов (`GET /`).
- При нажатии «Скачать» клиентский JS получает метаданные файла (из data-атрибутов).
- Клиент отправляет announce-запрос трекеру: `GET /api/announce?file_id=<id>&peer_id=<свой_peer_id>` .
- Трекер возвращает список пиров, имеющих данный файл (пока только сервер с `peer_id="server"`).
- Клиент устанавливает WebRTC-соединение с серверным пиром (используя сигнальный сервер Flask).
- Клиент запрашивает недостающие куски через DataChannel.
- Серверный пир (реализация WebRTC на стороне сервера) отправляет запрошенные куски.
- Клиент проверяет хэш каждого полученного куска (сравнивает с `piece_hashes` из метаданных).
- После успешной загрузки всех кусков клиент собирает их в Blob и предлагает сохранить файл.
- Клиент отправляет трекеру запрос на обновление статуса: `POST /api/peer_update` с `has_all_pieces=True` .
- Трекер создаёт/обновляет запись в `Peer` для этого пользователя и добавляет записи в `Piece` для всех кусков.

3. Скачивание вторым пользователем (Пользователь Б)

- Пользователь Б инициирует загрузку того же файла.

- Трекер на announce возвращает список пиров: `["server", "<peer_id_пользователя_A>"]`.
- Клиент устанавливает соединения с обоими пирами и скачивает куски параллельно с нескольких источников.
- Механизм проверки хэшей и становления сидом аналогичен шагу 2.

4. Поддержание актуальности пиров

- Периодически клиент может отправлять heartbeat-запросы трекеру для обновления `last_seen`.
- Трекер может удалять устаревшие записи пиров (неактивных более N минут).

---

5. Состав и структура проекта

```
p2pbigtorrent/
├── app.py                      # Точка входа Flask
├── requirements.txt            # Список зависимостей
├── models.py                   # SQLAlchemy модели
├── auth.py                     # Регистрация, логин, logout
├── torrent_tracker.py          # Реализация трекера (/api/announce)
├── file_manager.py             # Загрузка, разбиение, хэширование, удаление
├── config.py                   # Конфигурация (секретный ключ, путь к БД и
т.д.)
├── static/
│   ├── css/
│   │   └── bootstrap.min.css   # Bootstrap 5
│   ├── js/
│   │   ├── webtorrent_client.js    # Главный класс P2P-клиента
│   │   ├── webrtc_peer.js          # Управление WebRTC соединением
│   │   └── file_downloader.js      # Сборка файла из кусков
│   └── uploads/                    # Папка с физическими файлами
(сидирование)
├── templates/
│   ├── base.html               # Базовый шаблон
│   ├── login.html
│   ├── register.html
│   ├── index.html              # Список файлов (для пользователей)
│   └── admin.html              # Панель администратора (загрузка/удаление)
└── instance/
    └── app.db                  # Файл базы данных SQLite
```

---

6. Этапы разработки (для самоорганизации)

Этап	Содержание
1	Настройка окружения, создание каркаса Flask, подключение SQLAlchemy, Bootstrap.
2	Реализация моделей User, регистрация/авторизация.
3	Реализация моделей File, Peer, Piece, загрузка файлов администратором (разбиение, хэши).
4	REST API для получения списка файлов и метаданных.
5	Разработка самописного WebTorrent-клиента на JS (WebRTC, трекер-запросы, обмен кусками).
6	Реализация серверного трекера (`/api/announce`), хранение пиров и кусков в БД.
7	Интеграция клиента с API и трекером, отладка P2P-передачи.
8	Тестирование сценариев: админ загружает файл → пользователь А скачивает → пользователь Б скачивает с двух источников.
9	Доработка UI, индикаторов прогресса, обработка ошибок.
10	Написание пояснительной записки, презентации, подготовка к защите.

---

7. Критерии сдачи проекта

7.1. Состав сдачи

- Репозиторий Git (GitHub) с кодом и комментариями.
- Пояснительная записка (суть, архитектура, инструкция по запуску).
- Презентация (титул, идея, реализация, технологии, выводы).
- При необходимости — видео-демонстрация.

7.2. Критерии оценки (соответствие модулю)

- **Объём кода:** не менее 500 строк содержательного кода (Python + JS).
- **Чистота кода:** PEP 8, осмысленные имена, отсутствие copy-paste.
- **Качество проектирования:** классы, модули, разделение логики.
- **Применённые технологии:**
  - Bootstrap, ORM (SQLAlchemy), регистрация/авторизация, загрузка файлов, REST API, хранение в БД, хостинг.

    - **Самописный WebTorrent-клиент** (без готовых решений) — ключевое требование.
- **Оригинальность идеи:** P2P-ускорение внутри организации с контролем администратора.
- **Работоспособность:** реализованы все заявленные функции (админ загружает, пользователи скачивают P2P, трекер координирует).

---

8. Примечания для разработчиков

- **Запрещено использовать** любые готовые реализации BitTorrent/WebTorrent (например, библиотеки `webtorrent`, `bittorrent-tracker`, `torrent-piece` и т.п.). Всё пишется с нуля.
- WebRTC DataChannel работает без дополнительных серверов (ICE-серверы можно использовать публичные `stun:stun.l.google.com:19302`).
- Для упрощения можно ограничиться передачей только между браузерами в одной локальной сети (без TURN).
- Рекомендуется использовать `crypto.subtle.digest('SHA-256', data)` для вычисления хэшей на клиенте.
- Серверное разбиение на куски можно выполнять с помощью обычной работы с файлами в бинарном режиме.

Дата составления: 2025-04-17
Разработчики: Прошин Илья, Буслов Валерий